BÀI LUYỆN TẬP CẤU TRÚC LẶP

Bài 1: Tính tổng các số từ 1 đến N

Đề bài

Viết chương trình nhập một số nguyên dương `N`, tính tổng các số từ `1` đến `N` bằng cách sử dụng vòng lặp `for`.

Giải pháp

```
import java.util.Scanner;

public class TinhTong {
    public static int tinhTong(int n) {
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i;
        }
        return sum;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập số nguyên dương N: ");
        int n = scanner.nextInt();

        if (n > 0) {
            System.out.println("Tổng các số từ 1 đến " + n + " là: " +
tinhTong(n));
        } else {
            System.out.println("N phải là số nguyên dương.");
        }

        scanner.close();
    }
}
```

Phân tích

- **Sử dụng vòng lặp `for`** để lặp từ `1` đến `N` và cộng dồn vào biến `sum`.
- **Tách thành hàm `tinhTong(int n)`** để có thể tái sử dụng dễ dàng.
- **Kiểm tra điều kiện `n > 0`** để đảm bảo đầu vào hợp lệ.

Bài 2: Kiểm tra số nguyên tố

Đề bài

Viết chương trình kiểm tra một số nguyên dương `N` có phải là **số nguyên tố** không (số nguyên tố là số >1, chỉ chia hết cho 1 và chính nó).

Giải pháp

```
import java.util.Scanner;

public class KiemTraSoNguyenTo {
    public static boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập số nguyên dương N: ");
        int n = scanner.nextInt();

        if (n > 0) {
            if (isPrime(n)) {
                System.out.println(n + " là số nguyên tố.");
            } else {
                System.out.println(n + " không phải số nguyên tố.");
            }
        } else {
            System.out.println("N phải là số nguyên dương.");
        }

        scanner.close();
    }
}
```

Phân tích

- **Hàm `isPrime(int n)`** kiểm tra số nguyên tố:
  - Nếu `n < 2`, không phải số nguyên tố.
  - Kiểm tra chia hết từ `2` đến $\sqrt{n}$, nếu có số nào chia hết thì không phải số nguyên tố.
- **Sử dụng `Math.sqrt(n)`** giúp tối ưu thuật toán.

Bài 3: Tìm ước số của một số

Đề bài

Viết chương trình nhập vào số nguyên dương `N` và hiển thị tất cả **ước số** của `N`.

Giải pháp

```
import java.util.Scanner;
```

```java
public class TimUocSo {
    public static void lietKeUocSo(int n) {
        System.out.print("Các ước số của " + n + " là: ");
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập số nguyên dương N: ");
        int n = scanner.nextInt();

        if (n > 0) {
            lietKeUocSo(n);
        } else {
            System.out.println("N phải là số nguyên dương.");
        }

        scanner.close();
    }
}
```

Phân tích

- **Dùng vòng lặp `for` từ `1` đến `N`** để tìm các số chia hết cho `N`.
- **Tách thành hàm `lietKeUocSo(int n)`** giúp tái sử dụng dễ dàng.

Bài 4: In bảng cửu chương

Đề bài

Viết chương trình nhập một số nguyên `N` (từ 2 đến 9) và in ra bảng cửu chương của `N`.

Giải pháp

```java
import java.util.Scanner;

public class BangCuuChuong {
    public static void inBangCuuChuong(int n) {
        System.out.println("Bảng cửu chương " + n + ":");
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("Nhập số N (2-9): ");
        int n = scanner.nextInt();

        if (n >= 2 && n <= 9) {
            inBangCuuChuong(n);
        } else {
            System.out.println("N phải nằm trong khoảng 2-9.");
        }

        scanner.close();
    }
}
```

Phân tích

- **Vòng lặp `for` chạy từ `1` đến `10`** để nhân với `N`.
- **Hàm `inBangCuuChuong(int n)`** giúp tách logic, dễ tái sử dụng.

Bài 5: In dãy Fibonacci

Đề bài

Viết chương trình nhập vào số `N` ($N \geq 2$), hiển thị `N` số đầu tiên trong dãy Fibonacci.
Dãy Fibonacci: `0, 1, 1, 2, 3, 5, 8, ...`

Giải pháp

```
import java.util.Scanner;

public class Fibonacci {
    public static void inDayFibonacci(int n) {
        int a = 0, b = 1;
        System.out.print("Dãy Fibonacci: " + a + " " + b + " ");

        for (int i = 3; i <= n; i++) {
            int c = a + b;
            System.out.print(c + " ");
            a = b;
            b = c;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Nhập số N (N ≥ 2): ");
        int n = scanner.nextInt();

        if (n >= 2) {
            inDayFibonacci(n);
        } else {
            System.out.println("N phải lớn hơn hoặc bằng 2.");
```

```
        }

        scanner.close();
    }
}
```

Phân tích

- **Dùng hai biến `a, b`** để lưu hai số Fibonacci trước đó.
- **Dùng vòng lặp `for`** để tính các số tiếp theo.
- **Hàm `inDayFibonacci(int n)`** giúp tách logic.

Tổng kết

Các bài tập trên giúp luyện tập kỹ năng sử dụng **vòng lặp `for`, `while`, `do-while`** trong Java. Mỗi bài tập được **tách thành hàm** để có thể tái sử dụng và mở rộng dễ dàng.

BÀI LUYỆN TẬP THÊM

Nhóm 1: Cơ bản – Dùng vòng lặp for, while, do-while

1. **Tính tổng các số lẻ từ 1 đến N**
   - Nhập số nguyên `N`, tính tổng tất cả các số lẻ từ `1` đến `N`.
2. **Tính giai thừa của N**
   - Nhập số nguyên `N`, tính `N! = 1 × 2 × ... × N`.
3. **Đếm số lượng chữ số của một số nguyên**
   - Nhập số nguyên `N`, đếm số chữ số của `N`.
4. **Tính tổng các chữ số của một số nguyên**
   - Nhập số nguyên `N`, tính tổng các chữ số của `N`.
   - Ví dụ: `N = 123` → `Tổng = 1 + 2 + 3 = 6`.
5. **Đảo ngược số nguyên**
   - Nhập số nguyên `N`, in ra số `N` sau khi đảo ngược.
   - Ví dụ: `N = 12345` → `Kết quả = 54321`.
6. **Kiểm tra số đối xứng**
   - Nhập số nguyên `N`, kiểm tra `N` có đối xứng không.
   - Ví dụ: `121, 1331` là số đối xứng.
7. **Tính tổng các số nguyên tố nhỏ hơn N**
   - Nhập số `N`, tìm tổng tất cả số nguyên tố nhỏ hơn `N`.
8. **Liệt kê các số chính phương nhỏ hơn N**
   - Nhập số `N`, in ra tất cả các số chính phương `< N`.
   - (Số chính phương là số có dạng `m^2`, ví dụ: `1, 4, 9, 16...`).
9. **Tìm ước số chung lớn nhất (UCLN) của hai số**
   - Nhập hai số `A, B`, tìm UCLN của chúng.
10. **Tìm số Fibonacci thứ N**
    - Nhập `N`, tính số thứ `N` trong dãy Fibonacci (`0, 1, 1, 2, 3, 5, 8...`).

Nhóm 2: Ứng dụng vòng lặp để vẽ hình

11. **Vẽ tam giác sao rỗng**
    - Ví dụ: `N = 5`

```
*
**
* *
*  *
*****
```

12. **Vẽ hình vuông rỗng bằng dấu `*`**
    - Ví dụ: `N = 5`

```
*****
*   *
*   *
```

```
*   *
*****
```

13. **Vẽ tam giác vuông ngược**
    - Ví dụ: `N = 5`

```
*****
****
***
**
*
```

14. **Vẽ hình kim tự tháp**
    - Ví dụ: `N = 5`

```
    *
   ***
  *****
 *******
*********
```

15. **Vẽ số kiểu Pascal's Triangle**
    - Ví dụ:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Nhóm 3: Bài tập nâng cao – Tư duy thuật toán

16. **Liệt kê các số Armstrong nhỏ hơn N**
    - (Số Armstrong là số mà tổng lập phương của các chữ số bằng chính nó, ví dụ: `153 = 1^3 + 5^3 + 3^3`).
17. **Chuyển số từ hệ thập phân sang nhị phân**
    - Nhập số `N`, chuyển `N` sang hệ nhị phân.
18. **Tìm chữ số lớn nhất của một số nguyên**
    - Ví dụ: `N = 3297` → `Số lớn nhất = 9`.
19. **Tìm hai số Fibonacci lớn nhất nhỏ hơn N**
    - Nhập số `N`, tìm hai số Fibonacci lớn nhất nhưng nhỏ hơn `N`.
20. **Tính tổng dãy số theo công thức**
    - `S = 1 + 1/2 + 1/3 + ... + 1/N`.

Hướng dẫn học tập

- **Nhóm 1** giúp làm quen với các vòng lặp (`for`, `while`, `do-while`).
- **Nhóm 2** giúp ứng dụng vòng lặp để tạo hình học, rèn tư duy lặp lồng nhau (*nested loops*).
- **Nhóm 3** giúp rèn luyện thuật toán, tối ưu vòng lặp với `O(logN)`, `O(√N)`.